Production Architecture, the capstone

Session 4. 35 minutes, then 10 minutes to close.

The same stack, with nobody at the keyboard.

What changes when you stand up and walk away.

- The graph does not change. The trigger does.
- State has to live on disk. The chat will not be there in the morning.
- The budget stops being advice. Nobody is there to hit Ctrl-C.
- If you cannot read the last score, you cannot debug at 2 a.m.

Most agent failures are not model failures.

MAST, from 1,600+ traces across 7 frameworks:

Category	Share
System design and specification	**41.8%**
Handoff between agents	36.9%
Task verification	21.3%

Cemri et al., arXiv:2503.13657, NeurIPS 2025

Every one of those three is something you build, not something you buy.

Durable state. Five fields is enough to resume.

```
{
  "runs": 1,
  "last_gate": "pass",
  "last_reason": "the suite is green",
  "last_run_at": "2026-08-26T15:25:56Z",
  "loop": "fixer"
}
```

`.harness/state.json`, in the target repo, next to the receipt.

It survives the process. That is the whole point.

Observability is how you come back.

A trace is not a log file. Each span carries the tool name, the arguments, the output, the duration, the retries, and the error.

Then three numbers per run: **steps**, **loop count**, **cost per task**.

- Local JSON is production, if it is the record you actually open.
- Langfuse is that same record in a pane.
- A dashboard nobody reads is decoration.

The local gate and the remote gate must agree.

You met the push gate in Module 2. It reads `.harness/receipt.json` and refuses to push without a green one.

```
# .github/workflows/unattended.yml
on:
  workflow_dispatch: {inputs: {loop: {options: [fixer, implementer, enhancer]}}}
  pull_request:
  schedule: [{cron: "0 15 * * 1-5"}]
```

Exit **0** is a pass. Exit **2** is an escalation a human must read.

Exit **1** is a crash, and it is a different problem.

Same rule in both places, or the remote one is theater.

Lab 4. The Broken PR Fixer. 18 minutes.

```
cd labs/m4-fixer
claude -p "$(cat prompts/claude-code.md)"     # or codex, grok, opencode

task loop:fixer -- --doer reference
python -m loops.unattended --repo work/northwind-field-crm --loop fixer
```

Fill `loop.py`. Two functions: `summarize_failure` and `repair_until_green`.

Giving up is allowed. Giving up **silently** is the bug.

Falling behind is fine: `git checkout done-m4`.

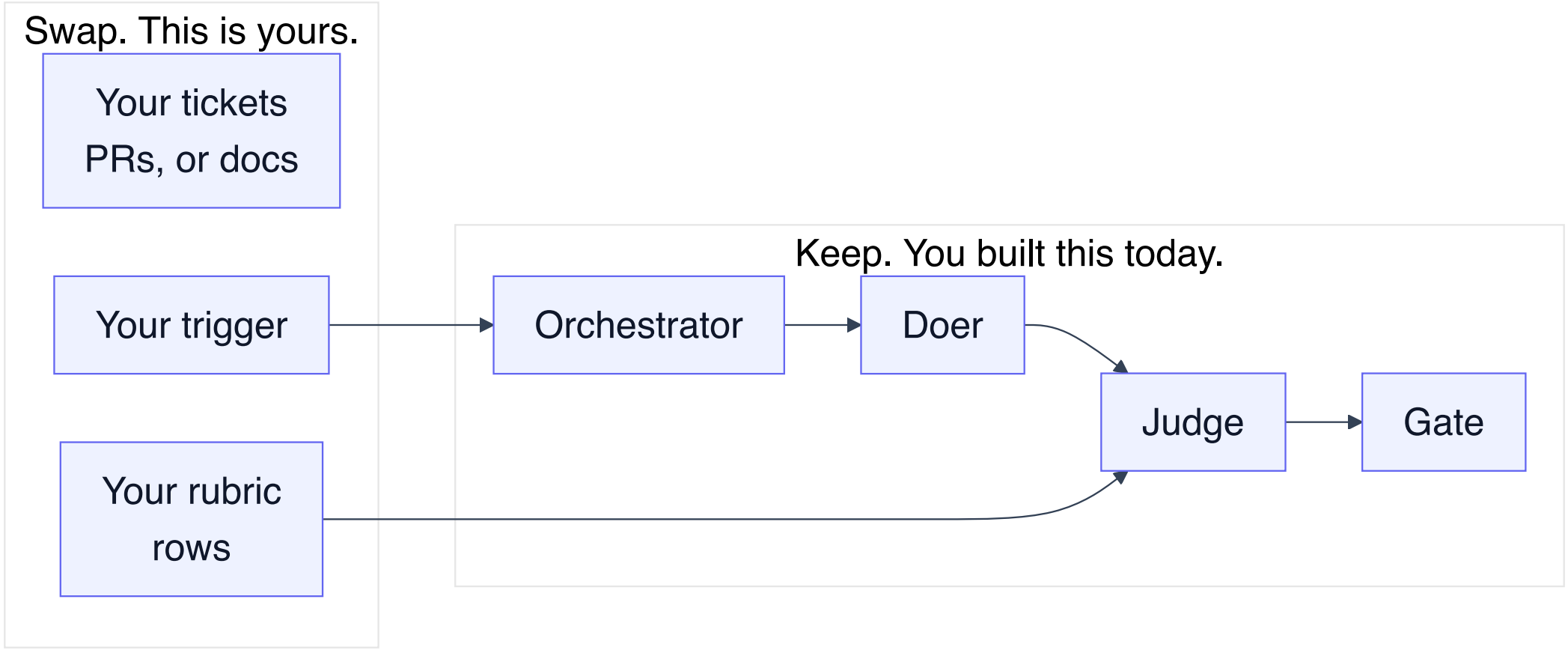
Why the receipt exists at all.

When a model may both act and verify its own work, it can produce **plausible false evidence**: invented test passes, file edits that never happened, fabricated API responses.

That is not hallucination about the world. It is a wrong judgment about the state of its own output, and a self-check cannot catch it by construction.

The receipt is not a convenience. It is the reason you can trust the run.

Swap the object. Keep the graph.



Four modules, one graph, four objects. That was on purpose.

Seven production loops. Named, not built.

Daily triage. PR babysitter. CI sweeper. And four more.

That list is a map home. It is not a second product to start on Monday.

The failure that gets you is slow.

A system passes every demo case, earns trust, then degrades over months with no single thing breaking. The causes are state, context, retrieval, latency, and observability. Not model capability.

So run the evaluation **weekly**, not quarterly. A 2% weekly drop is invisible in any one week and catastrophic across a quarter.

Close. 10 minutes.

Four artifacts, where they live, and what you do on Monday.

What you take home.

1. A running autonomous loop. Module 1.
2. A reusable evaluation harness. Module 2.
3. One research assistant over MCP. Module 3.
4. A production architecture. Module 4.

All four run on your machine, from a clean clone, with one `task setup`.

Where everything lives.

```
loops/                  the engine. It never imports the CRM.
labs/m1-enhancer        labs/m2-implementer
labs/m3-research        labs/m4-fixer
labs/takehome/          Claude Agent SDK and Deep Agents. Optional.
solutions/              the reference, and the code on these slides.
work/                   the target repo, cloned by task setup.
```

Branches `done-m1` through `done-m4` are green. Check one out any time.

What to do on Monday.

1. Pick **one** backlog object. Not five.
2. Write one ticket whose criteria a test can fail.
3. Give the target repo a `Taskfile.yml` that emits `junit.xml`.
4. Split the doer before you add a single tool.
5. Arm the push gate on day one, while the loop is still small.

Questions.

The loop is the product. The prompt is not.